

Sliding Window Technical Documentation

Analysis of Architectural Design, Common Implementation Pitfalls, and Standard Patterns

CORE PROBLEM ANALYSIS & CLARIFICATIONS

- **Sub-string Size Calculation:** You should compute the window/substring size using exactly `right - left + 1`. This accounts for zero-based indices and includes both boundary elements.
- **Shrinking Window Validation:** When shrinking the window, you must explicitly check if the element at the `left` pointer contributes to the desired state. If it does, safely decrement/remove its impact from your state tracking (e.g., frequency map or sum) before incrementing the `left` pointer.
- **Max Frequency Update Timing in Fixed Windows:** For a fixed window size, the maximum target value or global frequency tracking should only be evaluated and updated once the full window size `k` has been completed/established.
- **Global Max Updates:** Maximum values (like max tracking length) are traditionally updated outside or after the shrinking logic when you need to evaluate the optimal valid state across each independent index position.

1. First identify the type of sliding window

Fixed Size

- Maximum sum of size k
- Maximum vowels in substring of size k

Pattern:

```
expand
if(size > k) shrink
if(size == k) update answer
```

Variable Size

- Longest substring without repeating characters
- Minimum size subarray sum

Pattern:

```
expand
while(window invalid)
    shrink
update answer
```

Most mistakes happen because people use the wrong pattern.

2. Know what makes the window invalid

Ask:

"Under what condition should I move left?"

Examples:

- Duplicate character exists.
- Number of zeros exceeds k.
- Sum exceeds target.
- Distinct characters become greater than k.

Without defining the invalid condition, sliding window becomes confusing.

3. Distinguish between while and if

Use while

When multiple removals may be necessary.

```
while(freq[s[right]] > 1){
    freq[s[left]]--;
    left++;
}
```

Use if

When at most one element needs to be removed.

Fixed window:

```
if(right-left+1 > k){
    remove(nums[left]);
    left++;
}
```

Using if instead of while (or vice versa) is a very common mistake.

4. Update before removing or after removing?

For fixed windows:

```
if(size == k)
    answer = ...

remove left element
left++
```

Many people shrink first and lose information from the current window.

5. Don't confuse frequency with answer

Frequency map maintains state.

```
freq[s[right]]++;
```

Answer may be:

- `right-left+1`
- `maxFreq`
- `sum`
- `distinctCount`

These are different things.

6. Max-length and min-length problems update differently

Maximum length

```
while(invalid)
    shrink

answer = max(answer, size);
```

Minimum length

```
while(valid){
    answer = min(answer, size);
    shrink;
}
```

Notice the difference:

- Maximum length → shrink while invalid.
- Minimum length → shrink while valid.

This is one of the biggest sources of bugs.

7. Be careful with off-by-one errors

Always compute

```
size = right-left+1
```

not

```
right-left
```

8. Maintain exactly what the problem asks

Sometimes you don't need a map.

Examples:

- Maximum sum → Need: `sum`
- Maximum vowels → Need: `count`
- Longest substring without repeating characters → Need: `unordered_map<char, int>`

Using unnecessary data structures complicates the solution.

9. AtMost K trick

Many substring problems become easier by solving:

$$\text{Exactly}(k) = \text{AtMost}(k) - \text{AtMost}(k-1)$$

Examples:

- Subarrays with exactly K distinct integers.
- Count substrings containing all vowels and exactly K consonants.

Recognizing this trick is important.

10. Count problems are different from longest problems

Longest:

```
answer = max(answer, size);
```

Count:

```
answer += right-left+1;
```

Many beginners accidentally use `max()` when they should be accumulating.

11. Don't forget to remove frequencies completely

Wrong:

```
freq[s[left]]--;  
left++;
```

Right:

```
freq[s[left]]--;  
if(freq[s[left]] == 0)  
    freq.erase(s[left]);  
left++;
```

Otherwise: `freq.size()` will be wrong.

12. Sliding window works only when the window property is monotonic

Sliding window works because expanding or shrinking changes the condition predictably.

Works:

- ✓ Positive numbers sum $\geq$ target.
- ✓ Distinct characters $\leq k$.
- ✓ Number of zeros $\leq k$.

Doesn't work directly:

- ✗ Negative numbers with sum = k.
- ✗ Arbitrary non-monotonic conditions.

In those cases, prefix sum + hashmap is usually needed.

13. Don't start coding immediately

Before writing code, answer these five questions:

1. Fixed-size or variable-size?
2. What information should I maintain?
 - Sum?
 - Frequency map?
 - Distinct count?
 - Maximum frequency?
3. What makes the window invalid?
4. When should I shrink? `if` or `while`?
5. When should I update the answer?
 - When size becomes k?
 - After window becomes valid?
 - Before shrinking?
 - Inside shrinking?

THE BIGGEST MISTAKE PEOPLE MAKE

They memorize patterns instead of asking: "What does my window represent, and when does it become invalid?" Once you answer those two questions, 90% of sliding window problems become mechanical.